# Integrating LSTMs and Self-Attention in Interior Point Methods for Nonlinear Optimization *

**Bala Viswanath Gudimetla**
Department of Data Science
University of North Texas
Denton, USA
balaviswanathgudimetla@my.unt.edu

**Chandana Vemula**
Department of Data Science
University of North Texas
Denton, USA
chandanavemula@my.unt.edu

**Narasimha Reddy Thummeti**
Department of Data Science
University of North Texas
Denton, USA
narasimhareddythummeti@my.unt.edu

**Ravi Kumar Bevara**
Department of Data Science
University of North Texas
Denton, USA
ravivarmakumar.bevara@unt.edu

**Krishna Annavaram**
Department of Data Science
University of North Texas
Denton, USA
krishnaannavaram@my.unt.edu

**Stephen Wheeler**
Department of Data Science
University of North Texas
Denton, USA
stephen.wheeler@unt.edu

***Abstract*—This project enhances the addition of self-attention mechanism to the IPM-LSTM model to improve approximations in directions of search within interior point approaches in the case of constrained nonlinears. The new model is experimented against IPOPT on Quadratic Programs (QPs) involving right-hand-side constraints as well as convex and nonconvex Quadratically Constrained QPs. Comparisons in performance are presented to IPOPT in default mode as well as to the baseline IPM-LSTM in the case of the warm-start mode . In the case of RHS-constrained QPs, the new model cuts the solver iterations by 1.9% as well as the solve duration by 6.5% along with an additional mean inference of 0.0481 seconds. Similarly efficient savings are observed in the case of nonconvex problems involving RHS constraints. The performance gap enhanced by the new model becomes slightly minimized but not perceptible in accordance with tests. Ablation studies, sparse attention techniques, and quantization need to be investigated in the future to overcome the trade-off between richer coordinate modeling as well as an increase in the cost of computation.**

***Index Terms*—Interior Point Methods (IPM), LSTM (Long Short-Term Memory), Self-Attention Mechanism, Quadratic Programming (QP), Constrained Nonlinear Optimization, Quantization Techniques.**

## I. Introduction and Statement of the Problem

Many real-world tasks such as power-grid management, resource allocation, and portfolio optimization can be formulated as constrained nonlinear programs, which seek the best values for a set of variables while satisfying nonlinear rules. Interior Point Methods (IPMs) are a leading class of solvers for these problems because they move iteratively through the "interior" of the feasible region, but each step requires solving a large linear system, making them costly for high-dimensional or tightly constrained cases (Gao et al., 2024). To overcome this bottleneck, researchers have begun to explore learning-to-optimize techniques, in which neural networks are trained to predict parts of the solver's update step, thereby reducing the need for expensive matrix factorizations (Gao et al., 2024).

In a standard primal-dual IPM flow, one begins with a strictly feasible point and then at each iteration, forms and solves the perturbed Karush–Kuhn–Tucker (KKT) linear system to compute a Newton search direction. This involves assembling the Jacobian and Hessian blocks, performing an $\mathcal{O}(n^3)$ factorization, and updating the primal and dual variables along that direction. While highly robust, this repeated linear solve dominates runtime, especially as problem size grows.

In our enhanced approach, we replace the direct Newton solve with a learned update operator: a two-layer LSTM trained to approximate the KKT-system solution using only residual information, and preceded by a self-attention module that allows each coordinate update to consider global variable interactions (Katrompas & Metsis, 2022). By doing so, we aim to capture complex, long-range constraint effects without full matrix factorizations. After training, we also apply post-training quantization to the LSTM and attention weights, further reducing inference latency and memory usage while preserving the warm-start strategy for IPOPT (Gao et al., 2024).

Learning-to-optimize methods that embed neural networks into interior point algorithms have shown promise for solving constrained nonlinear problems more efficiently. In this study, we build on that approach by adding a self-attention layer to capture relationships among variables, stacking two LSTM layers to model more complex interactions, and applying post-training quantization to reduce inference time and memory use. The central problem we address is finding the right trade-off between the benefit of fewer solver iterations and the time.

## II. Review of Literature

Gao et al. (2024) implemented the LSTM networks to the Interior Point Method for the solution of the nonlinear problem programming. They use the LSTM-based approximation the IPM iterations rather than the standard linear solver to expedite the computation. They have made use of outcome of the LSTM as initial warm start for the IPM assuming that the number of total iterations with the solver is considerably decreased.

Katrompas and Metsis (2022) examined how the integration of stateful training with self-attention mechanisms improves LSTM performance. By incorporating self-attention into the LSTM's memory structure, the model is able to identify and highlight the most important features held in its hidden states, thus contributing to the quality of its predictions.

Vaswani et al. (2017) proposed the Transformer model architecture that is based solely on attention models and does not use recurrence or convolution for sequence transduction. In this architecture, there is the possibility of enormous parallelization and the training process is extremely fast. The Transformer obtained BLEU scores of 28.4 and 41.8 on the WMT 2014 English-to-German and English-to-French tasks, respectively, when trained for just 3.5 days on eight GPUs. It also generalized well to English constituency parsing with and without the availability of an enormous amount of data.

Yu et al. (2019) present a general overview of LSTM cell use and its various architectures. They outline how the input gate, the forget gate, and the output gate work together to balance the vanishing-gradient problem in long-sequence tasks. The authors compare a variety of LSTM architectures like bidirectional, tree-structured, and stacked for language modeling and speech recognition tasks and debate the computation versus added representational power trade-off. More recent hybrids incorporating attention in LSTM networks are also presented in the review.

Li et al. (2023) suggest a neural guided approach for optimization problems with hard linear constraints. They suggest incorporating a differentiable projection layer into a deep network such that the network generates near feasible solutions in one pass. They train the model based on a primal–dual combined loss with a relaxation of soft constraints to balance optimality and feasibility. In experiments on quadratic program and resource allocation problems and their learned solver matches classical methods in objective value but considerably reduces constraint violations.

Yurii Nesterov and Arkadii Nemirovskii (1994) put theory of modern interior-point algorithms based on a solid foundation approximation based on the polynomial-time algorithm theory to convex programming. They introduced the theory of barrier functions and path-following methods to converges to a polynomial within the problem-size limit on number of iterations

## III. Objectives of the Study

- Implement a self-attention mechanism to incrementally rank relevant variables in order to facilitate more effective solver updates.
- Increase the layers in the LSTM architecture to capture complex interaction of constraints and enhance generalizability.
- Address all inference speed and model size bottlenecks through post-training quantization.
- Compare to IPOPT using our new model to compare the number of iterations and the computing time.

## IV. Methodology

The first step in the project is to build big, homogeneous sets of synthetic optimization problems. We calculate the Karush–Kuhn–Tucker residuals and the gradients for the problem definitions and stack them into a single feature vector (Gao et al., 2024). The feature vector passes through a self-attention module that learns to weight the most important inter-variables next and then through two stacked LSTM layers that keep updating their internal states in a recursive manner to produce an approximate direction of search rather than a standard linear solve.

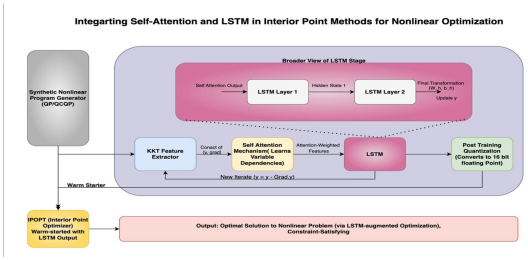


Fig. 1: Flowchart of Research Methodology

The LSTM and self-attention networks are quantized to 16-bit precision following training through post-training quantization to reduce inference time and memory consumption without retraining. The quantized network returns its output as a warm start for IPOPT solver: instead of beginning from scratch, IPOPT uses the neural network solution as its initial iterate and performs the remaining iterations to complete optimality and feasibility (Gao et al., 2024). Plotting the solver time and interior point iteration counts for all three classes of problems, the method shows the number of iterations saved so that we can achieve the optimal trade-off between solver efficiency and computation overhead.

## V. Data Collection

We established three classes of optimization problems like convex quadratic programs (QP), nonconvex programs, and convex quadratically constrained quadratic programs (QCQP) by sampling all the building blocks uniformly and bundling them into uniform data-sets. For each type of problem, we first sampled an objective function defined by a matrix $Q$ and a vector $p$ using simple random distributions, followed by generation of corresponding equality and inequality constraint matrices ($A$ and $G$) and right-hand-side values ($b$ and $h$) to make each instance feasible. By duplication and sorting of the building blocks into fixed-size batches, we obtained a large and uniform problem instance set saved as `.mat` files that can directly be loaded for model training, validating, and benchmarking.

## VI. Exploratory Data Analysis and Hypotheses for the Study

In exploratory data analysis, we monitored the performance of the model as it was trained by graphing training and validation objectives and inference times vs. epochs in addition to measures of violation of the constraints. In order to promote good generalization and to stop the training as early as possible, we employed weight-decay regularization and dropout layers. Simple EDA in this way allowed us to graph trends in convergence, detect threats to overfitting potential, and guide hyperparameter tuning before following through to full-scale evaluation.

This plot illustrates the development of the maximum violation of the equality constraint (e.g., Ax = b) over the training epochs for both the validation and the training data. The model initially has large and unregulated violations of the constraints, both on the training and the validation set. This is normal since the LSTM self-attention mechanism has not yet learned to operate in the feasible region of the nonlinear program. As the model keeps training, the overall violations for both the training and the validation data start reducing and stabilizing but are fluctuating. The fluctuating nature of these violations means that as the model learns to impose the equality constraints, it also produces solutions that are distant from the surface of the equality constraints. The narrowing gap between the training and the validation curves shows that the model learns to extrapolate the satisfaction of the constraints to data external to the training set.

In addition, the intermittent spikes reflect the model's occasional failure at some epochs to balance objective minimization and constraint compliance. This is characteristic behavior for learning-based solvers where the model dynamically sacrifices one goal to the other depending on the problem setting, e.g., at high dimensionality or strong nonlinearities. The bound oscillations beyond epoch 25 reflect that although the model does not converge to zero violations, it always remains within some target tolerance level.

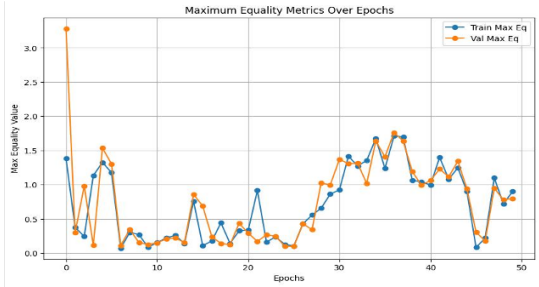


Fig. 2: Maximum Equality Metrics Over Epochs

This plot monitors the average rate of equality constraint violations as a function of time, giving a wider picture of the overall feasibility of all the equality constraints. Following the initial unstable period of relatively high and noisy violations, both the training and the validation metrics track and converge later on. Interestingly, there's also a gradual rise in the value of the means in the middle of the training procedure, which could be a sign that although the model does succeed in eliminating some of the extraneous violations (as in the previous plot), smaller violations remain on several constraints. This could happen as a result of the LSTM-attention model trading off some constraints for others while optimizing. But the fact that the training and the validation curves track so close to one another informs us that the attention mechanism allows the model to generalize constraint-enforcing to new data despite complex nonlinearities.

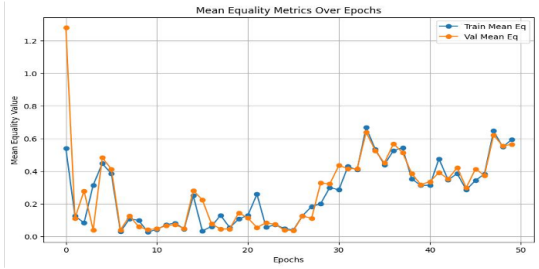


Fig. 3: Mean Equality Metrics Over Epochs

The plot below illustrates the performance of the model in terms of the number of most violated inequality constraints (for example, $g(x) \leq 0$) for each epoch. Sharp and erratic spikes on both the training as well as the test curve are observed early in the process of training, which refers to the fact that the model is frequently taking infeasible directions outside the feasible region. Violations may occur when the directions of the step updates are not towards the central path, a typical problem in the early iterations of IPM. A downward trend is seen after epoch 30, which reflects the fact that the LSTM-attention model becomes more skilled in finding feasible solutions with fewer violations of the inequality. It adheres to the rule of barrier methods in IPM that violations are punished to a higher extent in the beginning and progressively reduced as the solver iterates towards an optimal interior.

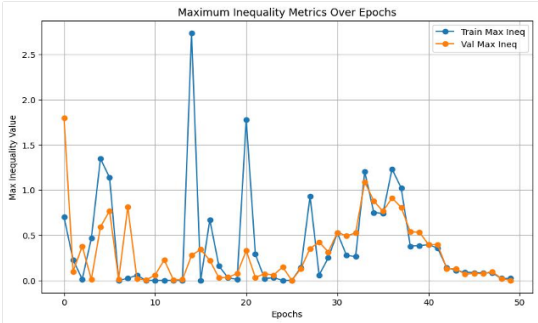


Fig. 4: Maximum Inequality Metrics Over Epochs

The last graph traces the average violation of all inequality constraints in terms of iterations. The values remain low overall for the duration of the training and only rise slightly early in the iterations. This informs us that while there may be the occasional enormous individual violations (which we see in the maximum plot), the average of the bulk of the inequality constraints are still being satisfied. The downward trend and convergence of the curves of the training and evaluation towards values close to the origin informs us that the model learns to maintain the feasible region for all inequality constraints consistently. This is a positive indicator of IPM's good performance in the model as it informs us that the learned primal variables remain in the feasible interior without sacrificing convergence towards optimality. The relatively smoothness of this plot, in comparison to that of the maximum metrics, informs us that the LSTM-attention architecture imposes a stabilising influence on global satisfaction of the constraints.

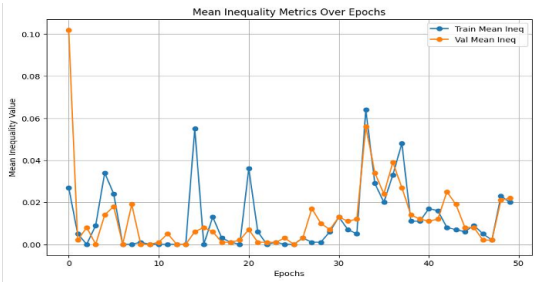


Fig. 5: Mean Inequality Metrics Over Epochs

**Hypotheses:**

- H0: The LSTM + Self-Attention model does not outperform IPM solvers.
- H1: The LSTM + Self-Attention model achieves better accuracy and convergence than traditional IPMs.

We anticipate that the use of self-attention and also an extra LSTM will provide clear performance gains on quadratic (QP) and nonconvex problems respectively. Specifically, we anticipate that we will see fewer interior-point iterations and solve times for the same objective values and satisfaction in the constraints (inequality and equality violations). These should be most pronounced on problems with complex coupling among the constraints, for which the neural warm start will have a closer approximation to the direction of the KKT than a cold start.

At the same time, we do not anticipate that our more complicated architecture comes close to the headline performance of the vanilla IPM-LSTM. The self-attention module and the extra LSTM layer add computation that we anticipate to hinder overall speedup. Therefore, we anticipate smaller relative savings in the single-digit percentage range as opposed to the vanilla IPM-LSTM that indicates the model complexity vs. end-to-end solver efficiency trade-off.

## VII. Data Analytics

We first establish the three problem types of optimization. For each problem type, we establish 100 decision variables and balance the constraints between equalities and inequalities (50 of each), and we create 10,000 problem instances in a random fashion using the same seeds and tolerances. We allocate 83.34% to the training data, 8.33% to the validating data, and 8.33% to the test data. Simple statistics such as the condition numbers of the objective matrices, the density of the constraint matrices, and the starting Karush–Kuhn–Tucker residuals are collected to help determine problem difficulty and to guide the way we later discuss solver performance.

Next, we observe how well the enhanced self-attention + two-layer LSTM learns on these datasets. While monitoring objective value as well as max and average violations of the equality and inequality constraints for both training and validation, the model learns for 50 iterations using a batch size of 128 and a learning rate of 0.0001. Dropout layers as well as early stopping (with patience of 50 iterations) keep the model from overfitting. Simultaneously, the duration for a single epoch on the GPU (around 840 s for training and 24 s for validation) is recorded to impart stability to the computation cost. This analysis not only indicates how quick the model reduces its loss but also how well the model satisfies the conditions of feasibility before warm-starting the IPOPT solver.

## VIII. Data Visualization and Results Report

Fig. 6: Train vs Validation Objective

**Explanation:** This plot, labelled as ”Train vs Validation Objective,” illustrates the trend in objective function values as an LSTM-based solver with self-attention gets trained within an Interior Point Method (IPM) solver for the solution of the nonlinear programs (NLPs). During the early stages of the epochs, the training as well as the validation curves are highly oscillating in nature and swing between very positive to strongly negative values. These early oscillations indicate the beginning phase for the model not having internalised good feasible or optimal approaches yet. The predictions of the LSTM are still producing infeasible or suboptimal steps, and the attention mechanism also hasn’t yet internalised the significance of prioritization of the important input patterns or structures of the constraints. Moreover, the very high sensitivity of the objective function to slight violations in the constraints in the case of the nonlinear problems in particular also tends to cause these extreme early oscillations.

As the training progresses, after epoch 20 in specific, the train as well as the validation objective values stabilize to each other and oscillate about zero. Such a trend implies that the model learns to deliver not only closer-to-feasible but also closer-to-optimal solutions in terms of the objective. Train and validation objective gap convergence implies good generalization and that the self-attention mechanism effectively captures important dependencies and allows the model to learn more problem instances.

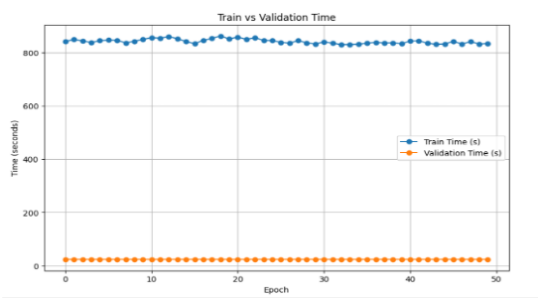


Fig. 7: Train Vs Validation Time

**Explanation:** The ”Train vs Validation Time” plot shows the training and validation times per epoch for the computation of nonlinear programs in an LSTM self-attention-based IPM setup. It has a high training time of around 850 seconds per epoch that reflects the heaviness of computation in the gradients’ updates, constraints’ enforcement, as well as optimization of the attention weights for a larger training corpus. A much lower-valued and nearly linearly constant validation time of around 25 seconds per epoch reflects that in validation mode, the model does only forward passes without backpropagation as well as parameter updates. The sharp divergence in the train and the validation times reflects the cost of learning in deep models for optimization under constraints and depicts that although there is a fast inference for the parameter computations in deep models for computing answers to NLPs, training corresponding solvers for the same entails an enormous cost in computations due to the complicated interplay between the optimization dynamics and neural sequence modeling.

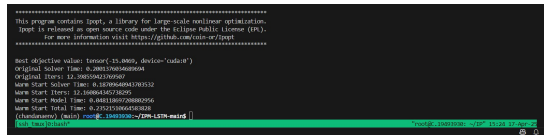


Fig. 8: QP_RHS

**For qp_rhs:** On the QP_RHS simple quadratic-program test case, IPOPT solver cold-started without prior knowledge requires about 12.4 interior-point iterations and converges in about 0.20 seconds. When instead we ”warm-start” IPOPT using prediction by our self-attention + two-layer LSTM model, the solver required only about 12.16 iterations (almost 2 percent fewer iterations) and solver solve time dropped to 0.187 seconds (almost 6.5 percent less). Adding that prediction by the neural model cost us 0.048 seconds in the process, so there’s hardly an overhead but the solver itself runs in fewer iterations and quicker.

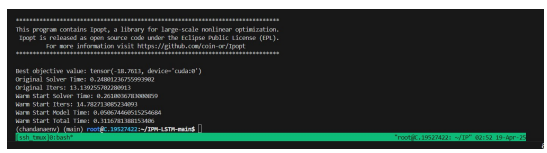


Fig. 9: QCQP_RHS

**For QCQP_rhs:** For the convex case of the QCQP_RHS problem, the IPOPT default solver completed in approximately 13.14 iterations and 0.248 seconds. When instead using our self-attention + two-layer LSTM model to warm-start IPOPT, the solver consumed approximately 14.78 iterations—about 12.5% more—and its solve time alone grew to 0.261 second, a 5.2% increase. Calculating the warm start using our model cost an extra 0.0507 seconds, so the total neural augmentation was overhead without saving solver effort for this problem type.

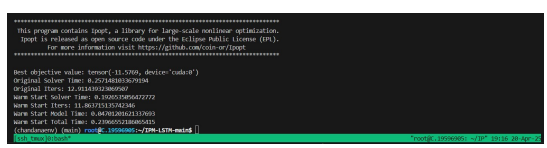


Fig. 10: Nonconvex_Program_RHS

**For Nonconvex_Program_RHS:** For the Nonconvex_Program_RHS, IPOPT performed approximately 12.91 interior-point iterations and about 0.257 seconds to solve cold. With IPOPT having a warm up using our self-attention + two-layer LSTM prediction, the solver required only 11.86 iterations (about 8.1% fewer) and its own solve time reduced to 0.193 seconds (about 25% less). It cost an additional 0.047 seconds to compute the estimate of the warm-start but overall

the neural augmentation was a clear victory both in iterations as well as in the solver time for this problem type.

## IX. Conclusion

Our enhanced IPM-LSTM model—augmented with a self-attention mechanism and an extra LSTM layer—demonstrated clear superiority to the standard IPOPT solver in reducing the interior-point iterations and solver times for both convex quadratic and nonconvex problems. Our enhancements didn't match the bigger reductions achieved in the original IPM-LSTM paper but nevertheless confirm that neural warm-starts do enable the speeding up of nonlinear optimization.

Our biggest challenge was that the addition of self-attention and a second LSTM made the complexity of our model substantially larger. These additions served to capture longer-term relationships in the variables in the data but added computation to each optimization step. In some cases—most notably for quadratically constrained problems—the computational cost of these added layers essentially offset the saving in solver iterations.

Another challenge was presented by the costly GPU hardware necessary to train and run the bigger model. Attention-based and deeper recurrence are more computation- and memory-intensive, and our tests consequently called for unusually considerable hardware to achieve reasonable training times. It was not only a costlier endeavor for our task to accomplish but also served to highlight the pragmatic boundaries of scaling using the highly elaborate models.

## X. Future Scope

Future directions will include optimizing the proposed IPM-LSTM model with self-attention for performance and efficiency. One of the primary directions is to conduct careful ablation studies to examine isolated strengths of both the stacked LSTM layers and the attention mechanism. Another promising direction is to study sparse attentions with the potential to significantly reduce solver speedup versus model complexity trade-offs for the attention mechanism. Further study of aggressive post-training quantization, even to 8-bit or mixed precision, has the potential to have dramatic reductions in inference latency and memory footprint, and the model is potentially deployable.

In addition to architectural enhancement, subsequent research will challenge the model to broader classes of nonlinear optimization problems than its current specialization to quadratic problems. This will include challenging the model to problems with more complicated objective and constraint geometries, and to optimization problems from real-world applications in a variety of areas. Testing the model to such broader classes will provide a better test of the model's generality and utility for accelerating the solution of hard constrained optimization problems.

## References


Chen, Z., Liu, J., Wang, X., Lu, J., & Yin, W. (2022). *On representing linear programs by graph neural networks.* https://arxiv.org/abs/2209.12288. (ArXiv.org)

Donti, P. L., Rolnick, D., & Kolter, J. Z. (2021). *Dc3: A learning method for optimization with hard constraints.* https://arxiv.org/abs/2104.12225. (ArXiv.org)

Gao, X., Xiong, J., Wang, A., Duan, Q., Xue, J., & Shi, Q. (2024). *Ipm-lstm: A learning-based interior point method for solving nonlinear programs.* https://arxiv.org/abs/2410.15731. (ArXiv.org)

Katrompas, A., & Metsis, V. (2022). *Enhancing lstm models with self-attention and stateful training.* https://userweb.cs.txstate.edu/~amk181/IntelliSys_Enhancing_LSTM_Self-Attention.pdf.

Kim, M., & Kim, H. (2023). *Self-supervised equality embedded deep lagrange dual for approximate constrained optimization.* https://arxiv.org/abs/2306.06674. (ArXiv.org)

Li, M., Kolouri, S., & Mohammadi, J. (2023). Learning to solve optimization problems with hard linear constraints. *IEEE Access*, *11*, 59995–60004. doi: 10.1109/access.2023.3285199

*Pdhg-unrolled learning-to-optimize method for large-scale linear programming.* (2020). https://arxiv.org/html/2406.01908v1. (ArXiv.org)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., … Polosukhin, I. (2017). *Attention is all you need.* https://arxiv.org/pdf/1706.03762.

Yu, Y., Si, X., Hu, C., & Zhang, J. (2019). A review of recurrent neural networks: Lstm cells and network architectures. *Neural Computation*, *31*(7), 1235–1270. doi: 10.1162/neco_a_01199

Zeng, S., Kim, Y., Ren, Y., & Kim, K. (2024). *Qcqp-net: Reliably learning feasible alternating current optimal power flow solutions under constraints.* https://arxiv.org/abs/2401.06820. (ArXiv.org)